

Extract and Load

Python for data engineering

Step 1a: Extract and Load - config.py

So.. I need to work with a source, extract and load the data to somewhere i can work with it in python.. Using python..

Will need:

- A config file

This should be where i describe the pipeline, and include things like connection information, this file doesnt move anything its just a desc. For the databases it will have all the connection information, the server, the port, the db name, for the files things like the paths and where the data should be loaded and maybe as well also specify the structure of the tables and the files.

What to learn to do this?

Data structures, this whole structure of the config file will be made up of dicts and lists, so for example the top layer a PIPELINE dict which will be full of key value pairs for example "database": { "port": 1234, "tables": [orders]}, etc etc. this are just purely examples and may not be exact key value pair or real contents of the config file

Config file will look like:

#Start

PIPELINE = {

"Layer: landing,

Run date: 01-01-26,

Database: {

Type: postgres,

Host db company internal

Port 1234

db : order_db

User: etl_user

password : 1234

Tables: orders

},

```

files: {
  "Format: csv
  Path: raw/files etc
  Header:true
},

"Paths:{
  Output: data / landing / orders
},

Load:{
  mode:FULL
},
Schema {
  "Order ID: strong
  "Amount" decimal; 10.2
  Country: string
  Created at: "timestamp"
},

"Quality :{
  "Required columns: [order_id, amount, country, created at],
  Not n_NULL: [order id]
}

}

```

Step 1b: Extract and load - ingestion_main.py

So now we need the main script to extract and load, as an example may need:

- Learning about loading from file, db, and kafka... this will be done using Spark & PySpark.
- Some scripting knowledge needed here will be the likes of Loops, for loops for the files, for the db etc as there will be a lot of files and tables needing loaded one by one.

- Also, working with files, Read + write (CSV + JSON + Parquet) ex.
Reading from a csv and writing into a parquet
- Getting data from Databases, connecting to a db and reading a table from it then writing it as a parquet file
- Same thing for kafka as db, reading from kafka and writing into a parquet.

So overall this main script will teach accessing (reading and writing) multiple sources and the file general structure will look like,

#Start

importing pyspark.sql from spark session, import config (our file) info like file paths db config, kafka config

Then creating a spark session

Files - for name path in file, paths in file paths spark read csv into write mode overwrite parquet

Db tables - for table in db config, spark read all the options, for table in db_config(tables) data = (spark.read .format .options .load()) data.write to parquet

Kafka - for topic in kafka config, data = (spark.read, .format, .option, .load()) write to parquet

spark.stop()

#end

Step 1c: extract and load - [logs.py](#)

Here also then a basic logging and error handling script just for some safety measures to be put into practice for the loading in the pipeline

#Start

Import logging

Logging basic config

Logger = get logger pipeline

Try:
Logger info
Row count
Logger info
Data.write mode appended parquet
Logger info

Except:
Exception as e:
logger.error(f"pipeline failed error = {e}")
Raise
#end

Here then also would need a second file
Data_quality.py

To check and catch these issues, there could be a lot of data quality checks that could be done in here but for example if data is empty the file looks like this:

```
#Start
#Check if dataset is empty
If len data.head(1) == 0:
    Raise value error("Landing quality failed, data empty")
Print landing file is not empty
```

Summary:

From this part of the project will touch core python topics like

- Data structures (lists + dicts)
- Control flow (loops and if / else)
- Working with files (csv, JSON, parquet)
- Connect to (DB, Kafka etc)
- PySpark basics
- Libraries (logging etc)
- Error handling basics

One point to note is that perhaps should try and expand this to touch upon apis as this area would be lacking, creating & building them, accessing them etc.

Data Cleaning

Step 2a: Data cleaning - cleaning_data.py

Here we're looking and tidying up the data and will be based on spark and pyspark basics again.

File will look like:

#Start

Imports from pyspark, session, sql.functions col, trim, lower, to date, configs paths and meta data

Def apply_text_cleanup(data, column):

Return data with column, column lower trim col column

Apply date cleanup

Same as above but to_date_column)))

Spark session get or create cleanup main

For table, schema in metadataitems

Data = spark read parquet landing paths table

For column in type in schema items

If type is string

Apply trim func

If column ends with _date

Apply date func

Data.write.mode overwrite parquyet silver path

spark.stop()

#end

Summary then, the topics touched on here:

- Control flow (loops and If / else)
- Data type functions (string, number, dates etc)
- Writing functions
- Working with files
- PySpark basics

Tab 3

Step 3a: Business logic - logic.py

Finally then, once data is clean can apply business logic and calcs or whatever is necessary like aggregations and stuff.

Need a file that looks like

#Start

Import from pyspark and configs

Spark start session business logic final

#Option 1 here is python & pyspark\

basic python functions nused here like aggregate etc with pyspark

Orders_data spark ready parquet SILVER_PATHS [orders]

Customers_data spark read parquet customers

Country level metrics

```
country _metrics_data = (  
Orders_data  
,join(customers_data, on customer id, how left  
Group by country  
.agg sum ("amount.aklias("total_revenue")
```

Country metrics data . write overwreite parquet
GOLD_PATHS ["Country metrics")

#Option 2 then is actual SQL and pyspark

orders segmentation

Orders_data.create or replace temp view ("orders")

```
Segmented_orders_data = spark.sql ("""  
SELECT country,  
Case when amount then  
When then  
Else Large etc  
End as order size  
... etc etc """)
```

Segmented ordered data . write . mode . overwrite . parquet (
GOLD_PATHS [order_ segments"]

Spark stop()

END

Summary:

Now then skills practiced and learned looks like:

- Working with files again
- Pyspark operations (like join aggregation, filter)
- Pyspark SQL